Grammar-Derived Operation Abstraction for Cross-Project Vulnerability Detection

Abstract

Learned vulnerability detectors often fail on software projects that were not in their training data. We ask how much of that failure depends on what each node of the program graph is told about the code. In a controlled comparison we change only this description: the identifier text itself; the same text with identifiers renamed; the name of the grammar rule that produced the node (the *grammar kind*); and a hand-built label for the operation a node performs. Detectors are trained on uniform random samples of two public corpora (BigVul and DiverseVul, about 10,000 functions each) and tested on whole projects that were held out, over ten matched folds, with paired statistical tests. Performance is measured by AUC, the chance that a vulnerable function is ranked above a benign one (0.5 is random). Text read from scratch ranks functions close to random (AUC 0.506 and 0.566 with the first network). Renaming identifiers or using grammar kinds raises AUC by roughly 0.09 to 0.20 on both corpora and under every architecture tried (three graph networks and a Transformer), and the two work about equally well. Much of that gain, however, disappears once function size is held fixed. The hand-built operation labels add nothing to the grammar kind, which we explain exactly: they are 97.7–98.4% predictable from it. Against reference points the picture is modest. Every graph model scores below a rule that ranks functions only by their length (AUC 0.736 and 0.715). On BigVul, models that never read what the code says do better still: gradient boosting over formatting statistics reaches AUC 0.896, because vulnerable functions carry trailing whitespace far more often than benign ones. Models that read raw source (a fine-tuned CodeBERT, a frozen UniXcoder probe) partly exploit this artefact, and their scores fall when formatting is removed.

With formatting removed, a fine-tuned CodeBERT beats the graph models by a small margin but not the length rule. The result is best read as a warning about baselines. We therefore report a controlled result about project vocabulary in learned detectors, not a stronger detector, and we release code, annotations (two language-model annotators) and scripts that regenerate every number.

Keywords: Program Representation, Static Analysis, Abstract Syntax Trees, Software Vulnerability Detection, Graph Neural Networks

---

1. Introduction

A learned vulnerability detector reads a function and outputs a score for how likely it is to contain a vulnerability; its quality is measured by AUC, the probability that a random vulnerable function gets a higher score than a random benign one (0.5 is chance). Many such detectors turn the function into a graph (its syntax tree plus a few extra links) and train a neural network over that graph. They report good accuracy when the test functions come from the same software projects as the training functions, and much worse accuracy on projects they have not seen [1, 2, 3]. Progress is usually claimed through new network architectures. This paper looks at a simpler factor that most papers hold fixed: *what each node of the graph tells the network about the code.*

Four ways to describe a node. Take the node for a call such as `av_malloc(n)`. Most detectors describe it by its **lexical token**, the text as written, here `av_malloc`. That text belongs to one project (FFmpeg allocates with `av_malloc`, the Linux kernel with `kmalloc`), so a network that learns from it meets mostly unknown words on a new project. We compare it with three descriptions that do not contain a project's vocabulary. An **anonymised token** replaces each identifier by a placeholder numbered by first appearance in the function. The **grammar kind** is the name of the grammar rule that produced the node (`call_expression`); it is fixed by the C language and costs nothing to obtain. An **operation type** is a hand-built label for what the node does (`av_malloc(n)`

is a memory operation); we built an eight-label version using lists of library functions and naming rules.

What we found, and how the result changed. In an earlier version of this work we argued that grammar kinds were the ingredient that mattered. Independent review, and the extra experiments it motivated, changed that conclusion. With graph, network and training held fixed and evaluation always on projects not seen in training, text read from scratch performs close to chance, and any of the three project-independent descriptions lifts AUC by roughly 0.09 to 0.20 on both corpora we use. But renaming identifiers works about as well as using grammar kinds, the hand-built operation labels add nothing to the grammar kind, much of the gain disappears when function size is held fixed, every network trails a rule that scores a function only by its length, and on BigVul a classifier that reads only formatting beats every network. We trace the last result to an artefact of the corpus that text-reading models, including a fine-tuned CodeBERT, partly exploit. The contribution is therefore a careful, partly negative empirical result and the methodology behind it, not a new detector.

Contributions.

- A **controlled comparison of node descriptions** across two corpora and four architectures on ten matched project-level folds, with explicit baselines for each competing explanation (text, text plus node type, anonymised text, and capacity-matched variants), direct paired tests, and per-project and function-size controls (Sections 4.8–4.12).
- **An exact explanation of why a hand-built operation label adds nothing** to the grammar kind, by two conditional entropies computed without training any model, offered as a check to run before an engineered abstraction of a parse tree is credited with an effect (Section 4.9).
- **A validation of the operation labelling as a static analysis** on the experimental corpora, against two blind language-model annotators with measured agreement (Section 4.7).

- **Controls that temper the claims**: AUC within bins of function size, a function-size rule, a formatting-only classifier and its effect on pretrained text models, which exposes a shortcut in BigVul (Sections 4.8 and 4.12).

Organisation. Section 2 reviews related work. Section 3 defines the program graph, the four node descriptions and the operation labelling, with a pipeline figure and algorithms. Section 4 sets out the corpora, sampling, class balance, cross-project protocol and statistics, and answers five research questions: the accuracy of the operation labelling (RQ1), which description transfers (RQ2), generality across corpora and architectures (RQ3), ablations (RQ4) and comparison with reference points (RQ5). Section 5 discusses implications and limitations, and Section 6 concludes.

2. Background and Related Work

2.1. Code Representations for Learned Detectors

Learned vulnerability detection has moved through three representations. Early sequence models treated a function as a token stream: VulDeePecker's code gadgets [4] and SySeVR's syntax- and semantics-based candidates [5] sliced programs into data-dependent token sequences for recurrent classifiers. The Code Property Graph (CPG) [6], unifying abstract syntax, control flow and data dependence in one structure, then made graph learning natural, and Devign [7] established the template still in use: parse to a graph, embed node text, propagate, pool, classify. Later work varied the graph or the propagation — inter-procedural slices in DeepWukong [8], feature-attention over program dependence in IVDetect [9], heterogeneous attention in [10]. A third line embeds code with pre-trained language models, either alone as in LineVul [11] or fused with a graph channel [12, 13].

What is largely invariant across this progression is the node feature: some embedding of the node's lexical content. Several detectors *do* also use node-type information. Devign initialises each node from its code text together with its node type [7], and ReVeal builds on the same construction [1]; Joern-based

pipelines expose node labels and edge types to the network. In those systems the type is an ingredient of a larger design and is never varied on its own, so one cannot tell from them how much of the benefit comes from the type and how much from the text. Our work isolates that choice. We hold the graph, the propagation scheme and the training protocol fixed and vary only what a node carries, including the Devign-style combination of text and node type as an explicit baseline (Section 3.5). We make no claim that using node types is new; the claim is about what a controlled comparison shows when they are used alone, with text, or with an engineered label set on top.

2.2. Normalisation and Abstraction of Code

Abstracting away identifiers is long-established practice, but as preprocessing rather than as an object of study. Clone detectors commonly normalise identifiers and literals so that renamed copies become textually identical; sequence-based detectors routinely map user-defined names to placeholder tokens (`VAR1`, `FUN1`) before training [4, 5]; the Devign corpus itself ships a normalised variant of every function.

Two properties distinguish what we do. First, existing normalisation is *anonymising*: it erases the identifier and replaces it with a fresh symbol, preserving only equality of occurrences. Ours is *semantic*: it maps a call onto the memory effect the callee is documented to have, so that `av_malloc` and `kmalloc` become the same symbol as `malloc` rather than three distinct placeholders. Second, and more importantly for a languages venue, we treat the mapping as an analysis with a specification and a measured error rate, rather than as an unexamined preprocessing step. We are not aware of prior work in this area that reports precision and recall for its own abstraction.

The idea of typing operations by their effect is of course not new in programming languages — it is the intuition behind effect systems and behavioural types — but those require type information and whole-program context that function-level vulnerability corpora do not provide. Our ϕ is deliberately weaker: syntax-directed, name-based, and total, so it applies to isolated, non-preprocessed func-

tions, at the cost of the precision quantified in Section 4.7.

2.3. The Cross-Project Generalisation Gap

The finding that motivates this paper is now well replicated. Chakraborty et al. [1] showed that detectors reporting strong within-project results degrade sharply on realistic, project-disjoint data; DiverseVul [14] demonstrated that simply adding training projects does not close the gap; PrimeVul [3] showed that much apparent progress reflects duplication and label noise, with F1 falling dramatically under a rigorous split; and [2] found the same degradation at repository scale.

Responses have been mostly architectural or adaptational — domain adaptation across projects [15, 16, 17], transfer with metric fusion [18]. Our study is complementary: it asks how much a node's description alone changes cross-project performance when everything else is fixed (Section 4.8), and how that change compares with the spread between architectures in a small family of networks (Section 4.10). It does not show that the gap is a representation problem; Section 4.8 shows that much of what a vocabulary-free description gains over text is explained by function size.

2.4. Evaluation Methodology in this Area

The methodological pitfalls we control for are documented in exactly the literature above, which is why we treat them as requirements rather than refinements. Duplication across and within corpora inflates results unless removed before splitting [3, 14]. Project mass is heavily concentrated, so a nominally cross-project split can degenerate into evaluation on a single repository unless large projects are handled explicitly. Threshold-dependent metrics are unstable when the calibration and test distributions differ, which they do by construction in cross-project evaluation. And functions within a repository are not independent, so confidence intervals computed over functions understate uncertainty. Section 4.4 states how each is addressed.

2.5. Federated and Privacy-Preserving Variants

A body of work applies federated learning to software engineering tasks, including defect prediction [19], code clone detection and defect prediction jointly [20], and vulnerability detection [21, 22], typically with token or metric features; FedProto [23] introduced sharing class prototypes rather than parameters, and differential privacy for learned models follows DP-SGD [24] with Rényi accounting [25]. We mention this line of work only to place the present paper against it. This paper makes no federated or privacy claim, reports no federated experiment, and evaluates a single centrally trained model throughout. A representation whose alphabet is fixed by the grammar is a plausible ingredient for such settings, since it carries no project-specific vocabulary, but we have not tested that and do not assert it.

3. Operation Abstraction

The object of study is a labelling function that replaces a program's project-specific lexical surface with a small alphabet of operation types fixed by the language grammar. This section specifies it, states the projection lattice relating its three granularities, and defines the graph on which it operates. Section 4 then evaluates it twice: as a static analysis, against annotated ground truth, and as a representation, by its effect on cross-project detection.

3.1. Overview and a running example

Figure 1 shows the whole pipeline. A C function is parsed into a syntax-tree graph. Each node of that graph is then described to the neural network by one of several *views*, and the views are the object of study. Everything downstream of the view (the network, the pooling, the training procedure) is the same in every experiment.

Running example. Consider the function body `n = len(buf); p = av_malloc(n * 4); if (!p) return -1;`. A node of its tree can be described in four ways. Its *lexical token* is the text written in the source (`av_malloc`, `n`, `buf`): this is a

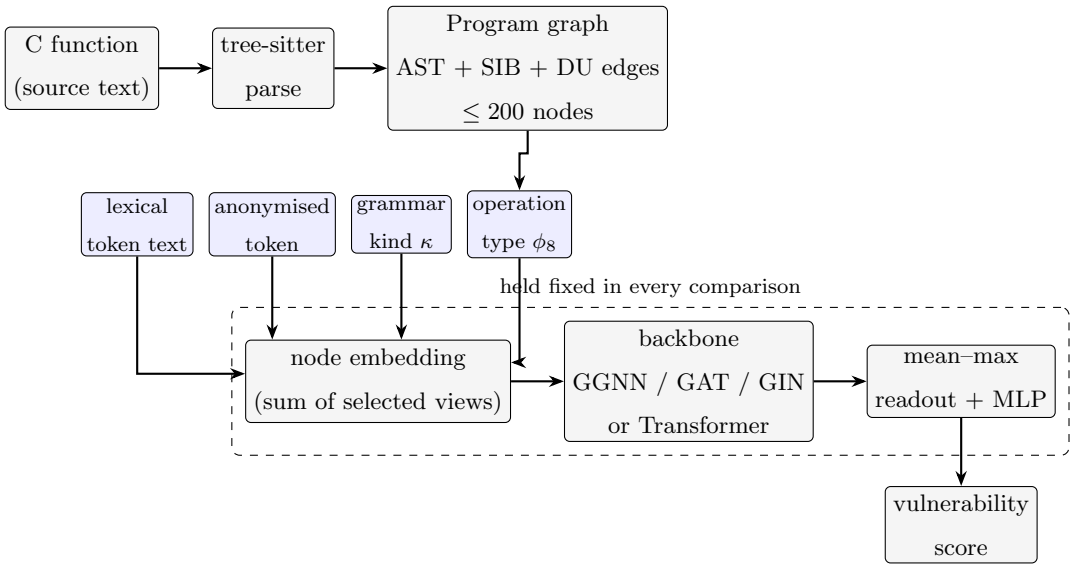


Figure 1: Pipeline. The only component that varies between compared systems is the choice of node view(s) (blue). The dashed region (embedding width, backbone depth, readout, optimiser, training budget) is identical across views. Evaluation always holds out whole projects.

property of the project that wrote it. Its *anonymised token* keeps only which names recur (`FUN_0`, `VAR_0`, `VAR_1`). Its *grammar kind* κ is the name of the grammar rule that produced it (`call_expression`, `identifier`, `binary_expression`), which depends only on the C language. Its *operation type* ϕ is a hand-built label for what the node does to memory or control flow (`av_malloc(...)` is a `MEMORY_ACCESS` node, `!p` a `COMPARISON`). Terms used throughout the paper follow these four definitions.

3.2. Program Graphs

Each function is parsed with `tree-sitter`, an incremental, error-tolerant parser that recovers a usable tree from non-preprocessed C. That property matters here: vulnerability corpora ship isolated functions without their headers, so a parser requiring a complete translation unit cannot be used. From the

resulting tree we build $G = (\mathcal{V}, \mathcal{E})$:

- $\mathcal{V}$: named AST nodes in pre-order, capped at 200;
- $\mathcal{E}_{AST}$: parent–child edges, both directions;
- $\mathcal{E}_{SIB}$: consecutive named siblings, giving source order;
- $\mathcal{E}_{DU}$: a def-use approximation linking successive occurrences of the same identifier leaf.

with $\mathcal{E} = \mathcal{E}_{AST} \cup \mathcal{E}_{SIB} \cup \mathcal{E}_{DU}$, giving on average 96 nodes and 247 edges per function in BigVul (110 and 285 in DiverseVul). We state plainly that this is a syntax graph with a data-dependence approximation, not a Code Property Graph [6]: interprocedural control flow and points-to-precise dependence are absent. Section 5.4 discusses what that bounds.

3.3. The Labelling Function ϕ

Let K be the node-kind alphabet of the tree-sitter C grammar ($|K| = 363$) and $\mathcal{T}_{32}$ the operation alphabet of Table 1. The abstraction is a total function

$$\phi : \mathcal{V} \longrightarrow \mathcal{T}_{32} \cup \{\bot\}, \tag{1}$$

where $\bot$ (`UNKNOWN`) labels nodes carrying no operation semantics. ϕ is deterministic and needs no name resolution, no build system and no whole-program context. It is defined by cases on the node's grammar kind:

1. **Call expressions** are labelled by their callee name through ψ (Section 3.3.1); an unrecognised callee yields `CALL_SITE`. User-defined functions, macros and calls through function pointers therefore all receive a label without resolution.
2. **Kind-determined nodes** follow a fixed table: `subscript_expression` → `ARRAY_INDEX`; `field_expression` → `FIELD_ACCESS`; `cast_expression` → `CAST`; `if`/`switch`/`case` → `BRANCH_*`; loops → `LOOP_*`; `break`/`continue`/`goto`/`return` → `JUMP_*`/`RETURN`; `assignment_expression` and initialising declarators → `ASSIGN`; `update_expression` → `ARITH_ADD`.

3. **Operator-determined nodes**: a `binary_expression` is labelled by its operator token ($+-$ → `ARITH_ADD`; $*/$ → `ARITH_MUL`; `%` → `ARITH_MOD`; bitwise → `BITWISE_OP`; shifts → `SHIFT_OP`; equality → `COMPARISON_EQ`; relational → `COMPARISON_REL`; logical → `LOGICAL_OP`). A `pointer_expression` is `PTR_DEREF` or `ADDR_OF` by its prefix operator; unary `!` is `LOGICAL_OP`.
4. **Otherwise** $\phi(v) = \bot$.

3.3.1. Callee classification ψ

ψ maps a callee name to an operation class in two stages.

Stage 1 (exact). A fixed allow-list of C standard-library names, grouped by the memory effect they carry: allocation, reallocation, release, block copy, block set, string copy, concatenation, buffer-writing formatting, length query, and stream I/O.

Stage 2 (component). Production C rarely calls those directly: FFmpeg wraps allocation as `av_malloc`, GLib as `g_malloc`, the Linux kernel as `kmalloc`, OpenSSL as `OPENSSL_malloc`. Stage 2 splits the callee identifier into components on underscore and camel-case boundaries and fires when a *whole component* matches an operation word.

Whole-component matching, rather than substring matching, is load-bearing. Substring matching — the obvious implementation — is measurably wrong on real code: it labels `__put_user` and `skb_put` (writes) as release operations via "put", `gen_new_label` as allocation via "new", `float64_is_zero` (a predicate) as a block set via "zero", and `reallocate_view` as reallocation, while *missing* `copy_to_user`, a genuine bounded copy. Section 4.7 quantifies the resulting error.

We further separate stream I/O from buffer formatting. `printf` and `fprintf` write to a stream and carry no caller-buffer overflow semantics; only the `s*printf` and `scanf` families fill a caller-supplied buffer. The distinction is not cosmetic: in our Devign sample `fprintf` and `printf` alone account for 655 call sites, every one of which a conflated rule would count as a memory operation.

Table 1: The $|\mathcal{T}_8| = 8$ operation alphabet and the $\mathcal{T}_{32}$ classes each abstracts. $\perp$ (`UNKNOWN`) labels nodes with no operation semantics: declarations, identifiers, literals, punctuation.

$\mathcal{T}_8$	abstracts ($\mathcal{T}_{32}$)	carries
MEMORY_ACCESS	MEMORY_{ALLOC,REALLOC,FREE,COPY,SET}, STRING_{COPY,CONCAT,FORMAT,LENGTH}	buffer lifetime, bounds
ARRAY_INDEX	ARRAY_INDEX	indexed access
PTR_DEREF	PTR_DEREF, ADDR_OF, FIELD_ACCESS, CAST	indirection
CONTROL_BRANCH	BRANCH_{IF,SWITCH}, LOOP_{FOR,WHILE}, JUMP_{BREAK,GOTO}, RETURN	reachability
ARITH_OP	ARITH_{ADD,MUL,MOD}, BITWISE_OP, SHIFT_OP	value computation
COMPARISON	COMPARISON_{EQ,REL}, LOGICAL_OP	guards
CALL_SITE	CALL_SITE, IO_CALL	unresolved or non-memory calls
ASSIGN	ASSIGN	value flow

3.4. The Projection Lattice

Three granularities are defined — $\mathcal{T}_{32}$ (finest), $\mathcal{T}_{16}$, $\mathcal{T}_8$ — related by surjections

$$\pi_{16} : \mathcal{T}_{32} \to \mathcal{T}_{16}, \qquad \pi_8 : \mathcal{T}_{16} \to \mathcal{T}_8, \tag{2}$$

each extended to fix $\perp$. Coarser labellings are compositions, $\phi_{16} = \pi_{16} \circ \phi$ and $\phi_8 = \pi_8 \circ \pi_{16} \circ \phi$, so the granularities are nested *by construction* rather than separately specified.

Proposition 1 (Nesting)**.** *For all $u, v \in \mathcal{V}$, $\phi(u) = \phi(v) \Rightarrow \phi_{16}(u) = \phi_{16}(v) \Rightarrow \phi_8(u) = \phi_8(v)$. Equivalently, the partition of $\mathcal{V}$ induced by ϕ_8 is coarser than that induced by ϕ_{16}, which is coarser than that induced by ϕ.*

Proof. Immediate from Eq. (2): ϕ_{16} and ϕ_8 are compositions of ϕ with total functions, and applying a function to equal arguments yields equal results. The induced partitions are the preimages of a chain of surjections, hence a chain of coarsenings. □

The consequence is that varying granularity varies exactly one quantity — the coarseness of a single labelling — so the granularity experiment in Section 4 is interpretable, which a comparison of three independently designed schemes would not be.

3.5. Node Views

A *view* is the symbol (or symbols) from which a node's learned embedding is looked up. Table 2 defines every view used in the paper. The definitions matter

Table 2: Node views. Leaves are tokens such as identifiers, literals and keywords; internal nodes are the syntactic constructs above them. "Invariant" means the symbol alphabet is the same for every project. Embeddings of combined views are summed.

View	Leaf node carries	Internal node carries	Alphabet	Invariant
`lexical`	source text ($\leq$ 20 chars, else <LONG>)	single token <INTERNAL>	training-fold tokens, at most 9,999 + <UNK>	no
`anon`	identifiers renamed by order of first appearance in the function, separately for variables, callees, fields, types and labels ($\mathtt{VAR}_0$, $\mathtt{VAR}_1$, $\mathtt{FUN}_0$, $\mathtt{FUN}_1$, $\mathtt{TYP}_0$, $\mathtt{LAB}_0$, ...; index capped at 15); numbers $\rightarrow$ NUM; strings and characters $\rightarrow$ STR; free-text leaves such as comments by their node kind; other named leaves (e.g. primitive type names) keep their text	<INTERNAL>	$\approx$ 1,000 symbols	monthly
κ	grammar kind	grammar kind	$\lvert\mathcal{K}\rvert + 1 = 364$	yes
o_8	operation type ($\perp$ if none)	operation type ($\perp$ if none)	9	yes
$\kappa \oplus o$	κ and o_8	κ and o_8	364 and 9	yes
`lex+`κ	lexical and κ	<INTERNAL> and κ	as above	no
`anon+`κ	anonymised and κ	<INTERNAL> and κ	as above	monthly

for fairness, because a lexical baseline that carries no information about node types would be compared against views that do. We therefore also include the combinations that real detectors use: Devign and ReVeal initialise a node with its code text together with its node type [7, 1], which corresponds to our `lex+`κ view.

The lexical view, precisely. Each leaf carries its exact source text. Every internal node carries the same token `<INTERNAL>`, so the plain lexical view contains *no* node-type information; it is the "text only" condition. Tokens enter a per-fold dictionary built *only* from the training projects (the at most 9,999 most frequent tokens occurring at least twice, plus `<UNK>`); every token not in that dictionary, including every token that exists only in held-out projects, is mapped to a single `<UNK>` embedding at test time. Out-of-vocabulary behaviour on unseen projects is therefore part of what the lexical view is measured on.

Why anonymisation is a separate baseline. Anonymising identifiers is the standard way to remove project vocabulary while keeping text (Section 2). The `anon` and `anon+`κ views isolate what grammar kinds add beyond that: if renaming identifiers already closes the gap, the grammar kind is not needed.

The operation view. Our default combined view is

$$\mathbf{x}_v = E_{\mathrm{op}}\big[\phi_8(v)\big] + E_{\mathrm{kind}}\big[\kappa(v)\big], \tag{3}$$

where $E_{\mathrm{op}}, E_{\mathrm{kind}}$ are learned tables with 9 and 364 rows. Both alphabets are fixed by the C grammar and the taxonomy, hence identical across projects and corpora.

One caveat, stated precisely. Node *features* contain no identifier, but ϕ *consults* identifier text to classify a callee, so a bounded amount of name-derived information (at most $\log_2(|\mathcal{T}_{32}| + 1) \approx 5$ bits per call node) influences the representation. What does not survive is the identifier itself, its spelling, or any project-specific vocabulary.

3.6. Networks and training, in brief

A *backbone* is the network that passes information between nodes. We use three message-passing networks over the program graph: a gated graph neural network (**GGNN**), in which each node updates its state from its neighbours with a gated recurrent unit; a graph attention network (**GAT**), which weights neighbours by learned attention; and a graph isomorphism network (**GIN**), which sums neighbour states and passes them through a small perceptron. A fourth, non-graph backbone is a **Transformer** encoder that applies self-attention over the nodes listed in *pre-order* (the order in which a depth-first walk first visits them) and ignores the edges. After the backbone, a *readout* pools the node vectors into one vector per function (the concatenation of their element-wise mean and maximum), and a small perceptron outputs the vulnerability score. Training minimises a binary cross-entropy in which vulnerable functions are weighted up by the benign-to-vulnerable ratio of the training set (capped at 20) *(class-weighted loss)*. "Def-use" edges connect successive occurrences of the same identifier, a cheap stand-in for data flow; "named nodes" are the syntax-tree nodes the grammar names (punctuation tokens are excluded).

3.7. Algorithms

Algorithm 1 gives the construction of a node's views; Algorithm 2 gives the evaluation protocol that every reported number follows.

Algorithm 1 Program graph and node views for one function

Require: C source f, node cap $N = 200$

$T \leftarrow$ TreeSitterParse(f)
$V \leftarrow$ first N named nodes of T in pre-order; record $n_{\text{full}} = |\text{named nodes}|$
for each $v \in V$ **do**
  $\kappa(v) \leftarrow$ grammar kind of v
  $\phi(v) \leftarrow$ **case** kind of v:
    call: ψ(callee name) {exact allow-list, then whole-component match}
    kind-determined: table lookup {rule 2 of Section 3.3}
    binary / pointer / unary: by operator token
    otherwise $\perp$
  if v is a leaf **then**
    lex(v) $\leftarrow$ source text of v; anon(v) $\leftarrow$ Rename(v) {first occurrence in f}
  else
    lex(v) $\leftarrow$ anon(v) $\leftarrow$ `<INTERNAL>`
  end if
end for
$\mathcal{E} \leftarrow$ AST edges (both directions) $\cup$ sibling edges $\cup$ def-use edges between successive occurrences of an identifier
return graph $(V, \mathcal{E})$ with views $\kappa, \phi, \text{lex}, \text{anon}$ per node

4. Experimental Design

4.1. Research questions

- **RQ1 (the labelling function as an analysis).** How accurately does ϕ identify memory and string operations, measured against independent blind annotation on the corpora used in the experiments?

- **RQ2 (which node view transfers).** With the graph, backbone and training budget fixed, how does cross-project detection change when the node view changes? In particular: does the grammar kind κ beat text;

Algorithm 2 Cross-project evaluation of one (view, backbone) system

Require: corpus sample D (uniform random, seeded), view m, backbone b

remove near-duplicates from D by their (κ, ϕ) node sequences
hold the projects whose size exceeds $0.3\times$ the mean fold train-only
deal the other projects into 5 folds, twice with different shuffles: 10 folds
for fold $k = 1, \ldots, 10$ **do**
    train set $\leftarrow$ all functions of projects not in fold k
    fit the lexical dictionary on the train set only
    initialise model with seed $1000+k$; train for 10 epochs with class-weighted loss
    score the held-out projects; store AUC, AUPRC and the raw scores
end for
compare two systems by paired fold differences (Wilcoxon) and a cluster bootstrap over held-out projects

does it beat text *plus* node type; does it beat anonymised text; and does the engineered taxonomy ϕ add anything to κ?

- **RQ3 (generality).** Does the effect hold across two corpora and across four architectures, including a Transformer?
- **RQ4 (ablations).** Which parts of the graph and of the taxonomy matter?
- **RQ5 (reference points).** How do the results compare with re-implementations of existing input representations, with a pretrained code model, and with a trivial function-size rule?

4.2. Corpora, sampling and class balance

We use BigVul [26] and DiverseVul [14], loaded from pinned public mirrors, plus Devign [7] for the analysis validation only. From each of BigVul and DiverseVul we draw 10,000 functions *uniformly at random* without replacement (a fixed seed, so the sample is reproducible), after which near-duplicates are removed (Section 4.3). This replaces an earlier design that took the first rows

Table 3: Corpus properties measured on our uniform random samples (10,000 functions drawn per corpus, then de-duplicated). *Prevalence* is the fraction of vulnerable functions in the whole sample; *Test prev.* is the pooled fraction over the held-out folds. They differ because the largest repositories, which have a lower share of vulnerable functions, are held train-only and never appear in a test fold (Section 4.4). *Truncated* is the percentage of functions with more than 200 named AST nodes, overall / vulnerable / benign.

Corpus	Functions	Vulnerable	Prevalence	Test prev.	Projects	Top-2 share	Nodes (mean)	Truncated (%)
BigVul	9,435	555	0.059	0.071	250	64.3%	96	18.1 / 42.3 / 16.6
DiverseVul	9,791	567	0.058	0.059	633	25.0%	110	25.6 / 53.4 / 23.9
Devign	9,666	4,455	0.461	—	2	100.0%	137	41.2 / 44.3 / 38.6

Table 4: Node-type distribution under ϕ_8 (BigVul sample). $\perp$ (`UNKNOWN`) nodes carry no operation label but keep their grammar kind.

Type	Share	Type	Share
$\perp$ (`UNKNOWN`)	75.23%	`COMPARISON`	2.72%
`PTR_DEREF`	6.56%	`ARITH_OP`	1.57%
`CALL_SITE`	5.45%	`ARRAY_INDEX`	0.53%
`CONTROL_BRANCH`	4.18%	`MEMORY_ACCESS`	0.30%
`ASSIGN`	3.47%		

in corpus order; it also raises the number of vulnerable functions from about 430 to 555 (BigVul) and 567 (DiverseVul). Table 3 gives the resulting properties.

Table 4 gives the distribution of operation types over the nodes of the BigVul sample: about three quarters of nodes carry no operation label ($\perp$) but keep their grammar kind, and memory operations are rare.

The data are strongly imbalanced, and we do not rebalance them. Only 0.059 of BigVul and 0.058 of DiverseVul functions are labelled vulnerable. We handle this in three ways. (i) Training uses a class-weighted loss, with positive weight equal to the training benign-to-vulnerable ratio capped at 20. (ii) Test sets are never resampled, so every metric reflects the true prevalence. (iii) We report AUC and AUPRC, which do not depend on a decision threshold, together with the trivial "everything is vulnerable" classifier as a floor; its F_1 at the observed

test prevalence is only $2p/(1+p)$.

Why overall prevalence and test prevalence differ. In BigVul the overall prevalence is 0.059 but the pooled prevalence of the held-out folds is 0.071. The reason is the train-only rule of Section 4.4: the largest repositories (Chrome and the Linux kernel; 6,070 functions, prevalence 0.052) are never tested, and the remaining projects have a higher vulnerable share. An earlier version of this paper quoted these two figures (0.054 and 8.8%) without connecting them.

Project concentration. Even after the largest repositories are held train-only, one project can dominate a test fold: in BigVul the repository `Android` makes up as much as 64% of a fold's test functions (the largest share in any DiverseVul fold is 20%). We therefore report every main BigVul comparison a second time with that project removed from the test predictions (Section 4.8).

The 200-node cap. A graph keeps the first 200 named nodes in pre-order. Table 3 shows that this truncates 18.1% of BigVul functions and 25.6% of DiverseVul functions, and that it is far from class-neutral: in BigVul 42.3% of vulnerable functions are truncated against 16.6% of benign ones (Fisher exact test $p < 0.001$). Vulnerable functions are simply longer. This matters in two ways, both examined below: a model may learn function length rather than anything about vulnerabilities (Section 4.12), and what lies beyond the cap is invisible to every compared system alike.

4.3. Deduplication

Near-duplicate functions are removed *before* any split. Vendored code appears under several project names, so without this step a held-out function can have a twin in training. Signatures are computed over the abstracted node sequences (κ and ϕ), which catches renamed copies that text hashing misses (the signature also includes the node count, which is capped at 200, so long functions that agree on their first 200 nodes are treated as copies; the first occurrence is kept whichever label it carries); this removes 2–6% of the sampled functions (the remaining counts are in Table 3).

4.4. Cross-project protocol

Evaluation uses project-level GroupKFold with ten folds in every experiment: projects are dealt into five folds twice, with two different shuffles, and no project is ever split between training and test. Projects too large to be placed without dominating a fold (those larger than 30% of the mean fold size: Chrome and the Linux kernel in BigVul, the Linux kernel only in DiverseVul) are held *train-only*. Every condition in every table is run on the same ten folds; the analysis scripts refuse to merge result files whose fold assignments differ. The two repeats are *not* independent: the splitter deals the projects out largest first, so the largest held-out projects fall into the same fold index in both repeats (for example the repository that dominates BigVul test folds appears in folds 0 and 5), while the small projects are reshuffled (the test project sets of fold k and $k + 5$ overlap by only 13–23% in Jaccard index). The ten folds are therefore better described as five folds with partial replication. Section 4.5 says how the tests account for this.

Per-fold training. The lexical dictionary is built from the training projects only (Section 3.5); the classifier is initialised and shuffled with seed $1000 + k$ on fold k, the same seed for every compared system. Hyper-parameters are in Table 5. They were fixed before the experiments and are not tuned on any fold; there is no validation split and no early stopping, so every system trains for exactly 10 epochs.

4.5. Statistical treatment

Every paired comparison is made between two systems on the folds both completed. The reported difference is the *mean over folds* of the per-fold difference in the metric, which is the quantity the per-fold columns of the tables average. We attach three kinds of uncertainty statement.

1. A **cluster bootstrap over held-out projects**. In each of the resamples, projects are drawn with replacement (a project is one cluster however many folds it is tested in); every function of a drawn project is repeated

as often as the project was drawn; the metric difference is recomputed inside each fold and averaged. The 95% percentile interval and a two-sided p follow. Functions inside a repository share style, APIs and often code, so resampling functions instead would understate the uncertainty. We use 1,000 resamples under each of five seeds for AUC (the verdict is called stable if the interval excludes zero under at least 90% of the seeds), and 500 resamples under one seed for AUPRC and the size-binned AUC.

2. A **Wilcoxon signed-rank test** [27], a paired test that uses only the ranks of the ten per-fold differences (exact, two-sided; its smallest attainable p is 0.002). Because fold k and fold $k+5$ are partial replicates of each other (Section 4.4), we also run it on the five repeat-averaged differences, where the smallest attainable p is 0.0625, so that test can never reach the 5% level on its own.
3. **Holm-adjusted p-values** [28], which correct a family of tests for the chance that one of them looks significant by luck. The family is all contrasts for one corpus and metric; adjusted values are stored in the result files and quoted where a conclusion depends on one contrast.

We therefore treat the cluster bootstrap as the primary inference, the Wilcoxon test as supporting evidence, and call a difference supported only if the bootstrap interval excludes zero. Whether truncation depends on the label is a question about counts, for which we use Fisher's exact test (a small-sample alternative to the chi-square test of independence).

4.6. Compute and reproducibility

All experiments run on CPU. One run of the BigVul GGNN grid takes roughly seven minutes per cell per core. Every table and every number in the text is generated from the stored per-fold predictions by scripts in the repository (`experiments/run_all.sh`); no result is typed by hand.

4.7. RQ1: ϕ as a static analysis

Design. An abstraction that is evaluated only by what it does for a downstream classifier can hide errors in its own labels. We therefore score ϕ's callee classification against annotated ground truth. This version of the validation differs from the one in the first submission of this work in three respects, each responding to a concern that a validation of this kind is only as good as its annotation.

- **The experimental corpora.** Callee names and one example call each are drawn from the *experimental samples* of BigVul and DiverseVul, not only from Devign; Devign (not used in the detection experiments) is kept so that the new labels can be compared with the first submission's. The sample is stratified over ϕ's predicted classes (up to 14 names per class, half from the most frequent names and half at random) plus 100 names drawn from callees that ϕ left unlabelled, so that recall is estimable. The Devign gold set of the original submission is annotated again.

- **Independent, blind annotation.** Each name is labelled by two annotators who see only the name, its frequency and one example call; they never see ϕ's prediction, each other's labels, or the analysis code. The criteria are written down in advance (`experiments/annotations/PROTOCOL.md`). The two annotators are language-model annotators of different capability, run as separate sessions. No human annotated these sheets.

- **Agreement and adjudication.** We report raw agreement and Cohen's κ [29] between the two annotators; every disagreement is resolved by a third blind pass that sees the two candidate labels in random order. The consensus is the ground truth for scoring. We also score against each annotator alone to show sensitivity.

Who annotated the original gold set. The 212 Devign labels of the first submission were assigned by a single annotator from written criteria. The sheet that annotator worked from listed ϕ's predicted class next to each name, so the annotation was not blind to the analysis under test, and the labels are stored

in a script in which every name not listed defaults to "none". That is a weaker protocol than the blind one above. Their agreement with the new consensus is $\kappa_C = 0.87$, so the original labels were largely sound but are not used for the headline scores here.

Tables 6 and 7 give the per-class scores and the annotation agreement.

Agreement. Table 7 reports agreement per corpus. Before adjudication the two blind annotators agree on 0.93 of the 675 names (pooled Cohen's kappa $\kappa_C = 0.91$; per corpus, raw agreement and κ_C: Devign 0.92 and 0.89, BigVul 0.94 and 0.93, DiverseVul 0.92 and 0.90), conventionally read as almost perfect agreement. 16, 13 and 18 disagreements respectively went to the third pass; in 1 Devign case(s) the third annotator agreed with neither of the first two and its label was used.

Accuracy of ϕ. Against the consensus, macro-averaged precision is 0.90 and recall 0.88 over the ten callee classes (Table 6); these are lower than the 0.976 reported for the original single-annotator labels, which we now regard as optimistic. The weakest class is `MEMORY_COPY`, with precision 0.63. Two caveats keep these figures honest. First, several classes have few positives (the smallest has 20), so their intervals are wide; Table 6 shows Wilson intervals [30] rather than only the point values. Second, the sample is stratified, so the numbers are per-name estimates, not a population-weighted accuracy.

What ϕ_8 actually needs. At $|\mathcal{T}| = 8$ every memory and string class collapses into a single `MEMORY_ACCESS` type, so confusing an allocation with a copy costs nothing. The question that matters is whether a callee is a memory or string operation at all. On that question precision is 0.94 and recall 0.94 pooled over the three corpora (0.95/0.95 on BigVul, 0.96/0.94 on DiverseVul).

Residual disagreements. The informative ones are structural, not noise. QEMU's `tcg_temp_free*` releases an intermediate-representation temporary rather than heap memory, so a release rule fires on it. Wrappers that allocate *and* copy or format (`g_strdup`, `g_strdup_printf`, `string_sprintf`) cannot be given a

single class by a single-label taxonomy. Project-specific I/O and teardown helpers (`bdrv_pread`, `ReadBlobMSBLong`, `RelinquishMagickMemory`) are missed because they match no allow-list entry and no whole-identifier- component rule. These are limitations of a name-based analysis with no type information; they bound what ϕ can claim and are one reason we do not expect it to beat the parser's own alphabet.

What the validation changed in the design. Matching whole identifier components instead of substrings removes writes and predicates mislabelled as releases and allocations (`__put_user`, `skb_put`, `gen_new_label`, `float64_is_zero`); separating stream I/O from buffer formatting keeps 655 `printf`/`fprintf` call sites in one corpus out of the memory class; and restricting the copy rule to explicit copy primitives raises its precision at the cost of recall. These three changes predate the present evaluation and are unchanged.

4.8. RQ2: which node view transfers across projects

In short: on projects never seen in training, a network that reads identifier text from scratch performs close to chance; removing the project vocabulary (by renaming identifiers or by using grammar kinds) lifts AUC by roughly 0.09 to 0.20; and the choice among the vocabulary-free views matters much less than the choice to leave the vocabulary out.

Everything except the node view is held fixed: graph, backbone (GGNN), depth, width, readout, optimiser, class weighting and gradient budget. Table 8 reports the BigVul results over 10 matched folds, and Table 9 the direct paired contrasts between views.

Text read from scratch does not transfer. With the lexical view the GGNN reaches AUC 0.506 on BigVul and 0.566 on DiverseVul, close to the 0.5 of a random ranking (across all lexical cells, 0.51 to 0.58). The cause is visible in the mechanism: after the per-fold cut, tokens that occur only in held-out projects map to `<UNK>`, and what remains of the vocabulary correlates with project identity rather than with the weakness.

Vocabulary-free views transfer. Replacing text with the grammar kind raises AUC by +0.185 (95% CI +0.130 to +0.236; bootstrap $p < 0.001$; Wilcoxon p 0.002; 10/10 folds won) on BigVul, and by +0.101 (+0.070 to +0.132) on DiverseVul. Renaming identifiers while keeping all other token text (*anonymised*) gives +0.202 (+0.149 to +0.249) on BigVul. The effect is large relative to the fold-to-fold spread of the lexical view (standard deviation 0.077 AUC; Table 8). After Holm adjustment over the whole family of contrasts on BigVul the bootstrap p of the κ-versus-text contrast remains < 0.001; the fold-level Wilcoxon test, whose smallest possible p is 0.002 before adjustment, reaches 0.225 after it, and 0.062 on the five repeat-averaged differences, so that test alone cannot carry the claim; the cluster bootstrap does.

The grammar kind is not better than anonymised text. This is the conclusion that changed most since the first version of this work. Reviewing it we added the baselines that a text-only comparison omits: anonymised tokens, and each of text and anonymised tokens combined with the kind (Section 3.5). The direct contrast of κ against anonymised+κ is -0.010 (-0.038 to +0.014) on BigVul and -0.012 (-0.030 to +0.002) on DiverseVul (Table 9). The same contrast against anonymised text alone (run on BigVul only) is -0.017 (-0.050 to +0.012). All intervals include zero. Identifier renaming, which is what sequence-based detectors have long done as preprocessing, therefore recovers essentially the whole benefit, to within the precision of this study. What the grammar kind offers is the same effect without any preprocessing choice, not a better effect.

Adding node type to text helps, but less than removing the text. The Devign-style combination of text with the node type improves on text alone by +0.086 (+0.038 to +0.111) on BigVul, so part of the benefit of structure is independent of removing names. But κ alone beats that combination by +0.100 (+0.064 to +0.156): while project-specific text is present, the network keeps fitting it. The comparison in the first submission, which pitted κ against a text view that carried no node-type information at all, was therefore partly "structure labels versus none"; Table 9 now separates the two.

The engineered operation taxonomy adds nothing to the kind. The direct paired contrast of $\kappa{+}\phi$ against κ is -0.004 (-0.017 to +0.021; Wilcoxon p 1.000) on BigVul and +0.008 (-0.005 to +0.022) on DiverseVul. The earlier submission inferred "no benefit" from the two views' separate gaps to the lexical baseline, which is not a test; here it is tested directly, and the interval contains zero in both corpora. On DiverseVul the fold-level Wilcoxon test gives $p = 0.027$ (8/10 folds favour $\kappa{+}\phi$), but the project-level interval contains zero and the repeat-averaged test cannot reach 5%, so we claim no effect; with the GIN the differences are +0.015 (BigVul) and +0.008 (DiverseVul), also without a supported effect. ϕ alone (ϕ_8) is +0.143 above text and +0.042 below κ (-0.004 to +0.067); the second difference is estimated imprecisely and we do not claim it. Section 4.9 explains why a null here is expected.

Capacity control. The kind table has 364 rows and the taxonomy 9, so one might suspect that κ wins by capacity. We therefore also trained κ squeezed through a nine-dimensional bottleneck, the width of the taxonomy table (`kind_low`). Its AUC is 0.708, and the difference from the unconstrained kind embedding is +0.017 (+0.004 to +0.040), so the bottleneck is, if anything, better. A capacity advantage therefore does not explain the result. (An earlier version of this paper reported a similar check without any code or data behind it; it has been replaced by this experiment.)

Controlling for function size: much of the gain disappears. Vulnerable functions are longer (Section 4.2), so a detector can gain AUC from length alone. A rule that scores each function by its node count reaches AUC 0.736 on BigVul and 0.715 on DiverseVul (second row of Table 8), higher than every GNN here. To ask what the networks learn beyond length we also compute the AUC *within* five bins of function size (pooled held-out predictions, bin edges from the held-out size quantiles); a score that carries no information beyond length has within-bin AUC 0.5. The picture changes substantially. On BigVul the within-bin AUC is 0.467 for text and 0.535 for the grammar kind, a paired difference of +0.069 (-0.014 to +0.112) that is not distinguishable from zero; the anonymised view

keeps a modest gain of +0.102 (+0.024 to +0.144). On DiverseVul the within-bin AUC of text and of the grammar kind are 0.523 and 0.523 (difference +0.001, -0.040 to +0.039), and no within-size difference between a GGNN view and text on DiverseVul exceeds 0.03 in absolute value or is significant. Across the corpus–architecture cells of Section 4.10 the grammar kind is never significantly better than text within size bins (it is not significantly better under any architecture, and the estimates for GAT and GIN are small). In other words, much of the large gap between vocabulary-free views and text in the overall AUC, and on DiverseVul all of it that we can measure, is a gap in the ability to represent how large or complex a function is; within a size bin the AUC of every view in the main grid lies between 0.47 and 0.88, that is, in a narrow band around chance. The intervals are wide, so a modest within-size gain cannot be excluded for the grammar kind, and we state only that it is not demonstrated. The vocabulary-free views plausibly help because node-kind counts, unlike text from a mostly out-of-vocabulary dictionary, let the pooled representation reflect function size; we have not tested that mechanism directly.

No single project drives the result. Because one repository can dominate a fold, we repeat the BigVul contrast with that project's test predictions removed: κ against text is +0.194 [+0.127, +0.241] without `Android`, compared with +0.185 with it. Table 10 gives the per-project AUCs for the held-out projects with at least ten vulnerable functions: the pattern of κ over text is present in most of them, although each single-project figure rests on few positives and is noisy.

4.9. Why the operation taxonomy adds little: an information-theoretic account

Section 4.8 reports that adding ϕ to κ buys little, and that ϕ alone is weaker than κ. Two different questions are involved, and both can be answered exactly, without training anything, because ϕ and κ are both functions of the same node. Over every node of the sampled functions of a corpus (before de-duplication, the first 200 nodes of each function) we compute their joint distribution and two conditional entropies. An entropy, in bits, measures how uncertain a symbol is; $H(A \mid B)$ is the uncertainty left in A once B is known.

- $H(\phi \mid \kappa)$ is the information in the taxonomy that the grammar kind does not already supply. If it is near zero, adding ϕ to κ cannot help.
- $H(\kappa \mid \phi)$ is the information in the grammar kind that the taxonomy destroys. If it is large, ϕ alone is a lossy copy of a representation that works.

The taxonomy adds almost nothing to the kind. $H(\phi \mid \kappa)$ lies between 0.05 and 0.06 bits on the three corpora (Table 11 and Figure 2); between 97.7% and 98.4% of nodes have an operation label that is fixed by the node kind (98.4% on BigVul). This follows from the definition rather than from an accident of the data: rules 2 and 3 of Section 3.3 are decided by the node kind or its operator token, so for most kinds ϕ is a constant function of κ. Only call expressions (is this callee a memory operation?) and binary expressions (which operator?) carry residual uncertainty.

The taxonomy discards much of the kind. $H(\kappa \mid \phi)$ is 3.10 bits on BigVul, 69% of the kind's entropy: collapsing 364 kinds into nine operation types throws away about two thirds of what the kind alphabet distinguishes. So ϕ is not a neutral re-encoding of κ; it is a lossy compression, and an abstraction in that position cannot be expected to outperform its source.

These are properties of two deterministic labelling functions over a joint table of $|\mathcal{T}_8| \times |K|$ cells, so they converge quickly: the bootstrap intervals over functions in Table 11 are at most 0.02 bits wide. The diagnostic costs minutes, needs no training, and would have predicted the experimental pattern of the next sections from the definition of ϕ alone. We offer it as a reusable check for anyone who proposes an engineered abstraction over a parse tree: compute $H(\text{abstraction} \mid \text{grammar kind})$ and $H(\text{grammar kind} \mid \text{abstraction})$ before crediting the abstraction with an effect.

4.10. RQ3: generality across corpora and architectures

In short: the benefit of dropping project vocabulary holds on both corpora and under every architecture we tried (the Transformer on BigVul only); the

choice of architecture moves AUC by much less than the choice of node view, but only within the family of small models we can train from scratch.

Table 12 repeats the main comparison on DiverseVul and under three further architectures: GAT and GIN (two other message-passing schemes) and a Transformer encoder over the pre-order node sequence, which uses no graph edges at all (BigVul only, for cost). Figure 3 plots the gains against the text-only view.

Corpora and message-passing schemes. Under GAT and GIN, and on DiverseVul, the grammar kind improves on text in every comparison (Table 12; for example +0.101 on DiverseVul with the GGNN, +0.094 with the GAT and +0.105 with the GIN). The gain varies by corpus (+0.185 on BigVul with the GGNN, against +0.101 on DiverseVul).

The architecture claim, narrowed. The earlier submission asserted that representation matters more than architecture on the strength of three message-passing networks whose AUCs differed by 0.027. That evidence cannot support a claim about architecture in general, and we now state what it does support. Under a fixed vocabulary-free view the three message-passing networks (GGNN, GAT, GIN) span 0.031 AUC on BigVul with the grammar kind and 0.027 on DiverseVul; under the text view they span 0.068 and 0.016. The widest of these ranges is smaller than the smallest gain of any vocabulary-free view over text (0.09), so within this family the node view is a larger lever than the backbone. The Transformer adds a different inductive bias (no edges); its AUC is 0.582 with text and 0.706 with the grammar kind, so the benefit of the vocabulary-free view is not specific to message passing. What our evidence cannot show is that no *pretrained* or much larger architecture would close the gap by itself, and Section 4.12 shows that a fine-tuned pretrained code model, reading format-normalised source, beats the vocabulary-free graph views by a small margin.

4.11. RQ4: ablations

We remove one component at a time under the grammar-kind view on BigVul (GGNN, the same 10 folds as before) and report the paired change in AUC (Ta-

ble 13). Beyond the node views of Table 9, which already ablate the contribution of text, anonymisation, kind and operation type, two structural components are examined.

Edge types. Removing the def-use edges changes AUC by +0.000 (-0.012 to +0.019); keeping only the syntax-tree edges changes it by +0.008 (-0.009 to +0.027). These measure the contribution of the data-flow approximation, and of source order together with it, on top of the tree. Neither changes AUC measurably under the grammar-kind view: the network gets little from the approximate def-use and ordering edges beyond the tree itself. (A variant that removes only the sibling-order edges was dropped from the grid for cost.)

Operation-alphabet granularity. Refining ϕ from eight to sixteen operation types changes AUC by +0.004 (-0.022 to +0.019) over 10 matched folds. The first submission reported a thirty-two-type run that rested on a single fold; the claim that granularity does not matter was not supported by that run and we do not repeat it. The one measurement we do have is consistent with a null, as expected from Section 4.9, since the classes that a finer alphabet separates are rare: the memory-operation class fires on only 0.30% of nodes (Table 4). The thirty-two-type refinement was not re-run for this version because of its cost, so we make no statement about it.

4.12. RQ5: reference points

Table 14 places our views next to re-implementations of the *input representations* used by existing approaches, a pretrained code model, and the trivial size rule, all on the same samples and folds. They are not reproductions of the published systems or their published numbers, which use different data splits and often within-project evaluation.

Devign/ReVeal-style input. Text together with the node type (`lex+`κ with a GGNN) reaches AUC 0.592 on BigVul and 0.607 on DiverseVul, which is below the grammar-kind and anonymised views in Table 8.

Pretrained code models: read the format-normalised rows. Table 14 holds two pretrained references: UniXcoder [31] with a frozen encoder (its weights are not updated; it turns each function into one vector) and a linear probe (a logistic regression trained on that vector), and a LineVul-style detector [11], that is, CodeBERT fine-tuned end to end on the function text (function-level classifier only; three epochs, the same class-weighted loss and folds as every other system, run on a rented GPU). On raw source they reach AUC 0.705 and 0.922 on BigVul, 0.623 and 0.693 on DiverseVul. On BigVul those figures partly measure the formatting artefact of Section 4.13, so the fair comparison uses format-normalised source, where the same two systems reach 0.645 and 0.728 (BigVul) and 0.604 and 0.705 (DiverseVul). Against these normalised references the vocabulary-free graph networks are better than the frozen probe (GGNN with κ: +0.047 on BigVul, +0.003 to +0.091; +0.063 on DiverseVul) and below the fine-tuned model by a small margin (-0.037, -0.068 to -0.018 on BigVul; -0.038, -0.056 to -0.023 on DiverseVul). Two lessons follow. First, "text hurts cross-project transfer" is a statement about token embeddings learned from scratch on a few thousand functions, not about text in general: a model that has seen billions of code tokens before, and is fine-tuned, uses identifiers and beats our graph models by about 0.04 AUC. Second, that margin is small in absolute terms: the fine-tuned model is not distinguishable from the function-size rule (BigVul -0.008, -0.026 to +0.006; DiverseVul -0.010).

A size rule beats every network. Scoring a function by its node count alone gives AUC 0.736 (BigVul) and 0.715 (DiverseVul), a higher mean AUC than any network in this study. (On BigVul a formatting-only classifier is higher still; Section 4.13.) The paired differences are significant under the project-cluster bootstrap for most networks (e.g. κ with the GGNN against size: -0.045, -0.086 to -0.022 on BigVul) but not for all (for example the GIN with $\kappa+\phi$ on BigVul). We read this as a caution about the whole benchmark family: under project-disjoint evaluation, much of what these corpora reward is a length prior, and absolute AUCs of 0.6 to 0.7 should not be read as evidence of vulnerability

understanding.

4.13. A formatting artefact in BigVul, and what it does to models that read text

In short: in BigVul, the raw formatting of a function tells whether it is labelled vulnerable, almost independently of what the code does. A model that reads source text can use this; our graph models cannot, because the parser discards formatting.

How it was found.. We fine-tuned a LineVul-style detector (CodeBERT [11] fine-tuned on the raw function text, on a GPU rented for the purpose, with the same ten project-level folds) as the stronger pretrained reference that reviewers of an earlier version asked for. It scored a mean AUC of 0.922 on BigVul (AUPRC 0.748), far above every graph model, which is implausible for cross-project detection and prompted a search for a shortcut.

The artefact.. 61% of the vulnerable BigVul functions in our sample contain at least one line that ends in trailing whitespace, against 2% of the benign functions. In DiverseVul the corresponding figures are 11% and 4%. Two baselines that never read what the code says expose the consequence (Table 14, last rows). A single rule that scores a function by the fraction of its lines with trailing whitespace reaches AUC 0.830 on BigVul (0.542 on DiverseVul); gradient boosting over sixteen formatting and size statistics (length, number of lines, tabs, carriage returns, trailing whitespace, blank lines, comment markers, indentation, and so on) reaches 0.896 (0.634 on DiverseVul), on the same project-disjoint folds. The first is above every graph model in this study; the second is above all of them by a wide margin. We do not know why the corpus carries this signal (a plausible cause is that benign functions pass through a different extraction or formatting step than vulnerable ones), only that it is present, strong, and project-independent.

Removing it.. We re-run the text-reading systems on *format-normalised* source: the parser's token stream with comments dropped and every token separated by

one space, so indentation, line breaks, trailing whitespace, tabs and comments are gone while identifiers, literals and operators are unchanged (0% of normalised functions contain trailing whitespace). Table 14 shows that fine-tuned CodeBERT falls from 0.922 on raw BigVul source to 0.728 on normalised source (paired difference +0.194, 95% interval +0.141 to +0.224; AUPRC from 0.748 to 0.189), and the frozen UniXcoder probe from 0.705 to 0.645 (+0.060). On DiverseVul, where the artefact is much weaker, the corresponding falls are from 0.693 to 0.705 and from 0.623 to 0.604. Even after normalisation, fine-tuned CodeBERT reaches only 0.728 on BigVul and 0.705 on DiverseVul, against 0.736 and 0.715 for the function-size rule (BigVul paired difference -0.008, -0.026 to +0.006): a fine-tuned 125-million-parameter model reading the identifiers does not clearly beat a count of syntax-tree nodes.

What this means for the rest of the paper.. First, the graph models are not affected: the grammar kind, the anonymised and the operation views are computed from the parse tree, which contains no formatting, so the comparisons of Sections 4.8–4.11 stand. The function-size rule and the size-binned AUC are likewise unaffected, since node count does not depend on formatting. Second, the pretrained references of Section 4.12 must be read on normalised source, not raw: on raw BigVul source they partly measure formatting. Third, for the field, any result on BigVul from a model that reads raw text, including the large pretrained models, should be checked against a formatting-only baseline before it is attributed to an understanding of the code. We have not audited published results and make no claim about them; the check costs one line of code per corpus.

4.14. Summary of findings

1. **Text read from scratch fails across projects.** AUC is 0.506 (BigVul) and 0.566 (DiverseVul), close to random.
2. **Removing project vocabulary helps by about 0.09 to 0.20 AUC,** on both corpora and under every architecture we ran (the Transformer on

BigVul only), whether it is removed by anonymising identifiers or by using the grammar kind.

3. **The grammar kind does not beat anonymised text;** its advantage is that it needs no preprocessing decision.
4. **The engineered operation taxonomy adds nothing** to the grammar kind, and the reason is measurable: it is almost a function of it (Section 4.9). As a static analysis it is accurate for the question it answers (memory operation or not: precision 0.94, recall 0.94) but less accurate than first reported at the level of individual classes.
5. **Much of that gain is about function size.** Within bins of function size the gain over text is not distinguishable from zero for the grammar kind in every corpus–architecture cell we ran, and within-bin AUC is 0.535 (BigVul) and 0.523 (DiverseVul).
6. **Classifiers that read no code beat every network:** a one-line size rule on both corpora and, on BigVul, a classifier on formatting statistics alone (AUC 0.896), because the corpus has a formatting artefact. Models that read raw text partly exploit it (Section 4.13). Absolute performance of the graph models is weak.

5. Discussion

5.1. What the evidence supports

Four statements survive every control we ran. First, a graph network that learns embeddings of project-specific text from scratch transfers to new projects barely better than chance. Second, any description of nodes that does not contain that vocabulary does substantially better, by roughly 0.09 to 0.20 AUC, on both corpora and under every architecture we ran. Third, among the vocabulary-free descriptions the differences are small: renaming identifiers is about as good as the grammar kind, and an engineered operation taxonomy adds nothing to the grammar kind. Fourth, much of the gap between text and the vocabulary-free views is a gap in representing function size: within size bins

the grammar kind is not distinguishable from text, and on DiverseVul no GGNN view is. The first two statements are consistent with a simple account: a lexical model's input at test time is largely out-of-vocabulary, and the part that is in vocabulary correlates with project identity rather than with the weakness. We have not isolated the mechanism experimentally (for instance by varying the out-of-vocabulary rate while holding everything else fixed), so this remains an interpretation.

5.2. What it does not show

The results do not show that grammar kinds, or any node description, make vulnerability detection work across projects. A rule that ranks functions by their node count beats every network in the study (Section 4.12), and on BigVul a classifier that sees only formatting beats all of them (Section 4.13), and the gains of the vocabulary-free views over text largely vanish within bins of function size (Section 4.8). Nor do they show that text is harmful in general: a fine-tuned pretrained code model reading format-normalised source beats the graph models by a small margin (Section 4.12). The honest reading is that the standard from-scratch lexical baseline of graph-based detectors is weak under project-disjoint evaluation, so improvements over it are cheap and should be compared with the stronger baselines listed above.

5.3. Evaluating an abstraction before crediting it

For a languages audience the methodological point is the more portable one. A proposed abstraction over a parse tree should be examined on two fronts that are rarely both reported. As a program analysis, it should be scored against annotated ground truth with agreement and uncertainty reported, as in Section 4.7; the first version of this work did that with a single, non-blind annotator and reported an accuracy higher than the blind protocol supports. As a representation, it should be compared with the parser's own alphabet through the two conditional entropies of Section 4.9, which cost minutes and would have predicted our null from the definition of the abstraction. A third check is cheap

and is the one we most nearly omitted: ask whether a function-length rule does as well.

5.4. Threats to validity and limitations

Graph fidelity. Our graphs are tree-sitter syntax trees with sibling and approximate def-use edges, not Code Property Graphs. Every compared system consumes the same graph, so the comparison is internally fair, but absolute performance is bounded by this choice and conclusions may differ with richer graphs.

Sample, truncation and length. Each corpus is a uniform random sample of about 10,000 functions (roughly 555 and 567 vulnerable functions). The 200-node cap truncates a quarter or fewer of functions overall but far more of the vulnerable ones (Table 3), so what lies beyond the cap is invisible to every system. Because vulnerable functions are longer, any detector may partly learn length; we measured this rather than removing it.

Training budget and tuning. Every system trains for the same fixed number of epochs with hyper-parameters chosen in advance and not tuned on any fold (Table 5). A longer or tuned schedule could change absolute numbers, in particular for the lexical view, although fixing the budget across views is what makes the comparison controlled. Each fold uses a single initialisation seed shared across systems, so run-to-run variance within a fold is not estimated; the paired tests treat folds, not seeds, as the unit.

Dependence between folds. Folds from the second repeat reuse some test projects of the first, so the ten folds are not independent. The cluster bootstrap resamples projects and the Wilcoxon test is reported alongside it, but the effective sample size is smaller than ten.

Annotation. The callee validation uses two blind annotators and a third pass on disagreements, but all three are language models; no human labelled the sheets. The original, single-annotator labels agree with the new consensus at Cohen's κ of 0.87. A human second annotator remains desirable, and the released sheets allow one to be added. Several classes have few positive names,

so their intervals are wide.

Label noise and language. We inherit the labels of the corpora, whose accuracy is contested in the literature that produced them, and we study C only.

Pretrained references. The fine-tuned model reproduces only the function-level classifier of LineVul, for three epochs, without checkpoint selection; the UniXcoder result is a frozen encoder with a linear probe on 256 tokens. Longer or tuned schedules, or larger models, could score higher.

Earlier claims withdrawn. The first submission also reported federated and privacy experiments and a learned edge mask. They are not part of this work, and nothing here supports or refutes them.

6. Conclusion

We asked how much of the cross-project failure of graph-based vulnerability detectors depends on what a node is told about the code. Under a controlled, project-disjoint protocol with ten matched folds, node descriptions that contain a project's vocabulary performed close to chance, while descriptions that do not (renamed identifiers, or the parser's grammar kinds) performed better by roughly 0.09 to 0.20 AUC on two corpora and every architecture we ran. They performed about equally well, and the hand-built operation taxonomy that we expected to help added nothing to the grammar kind. We explained that null exactly: the taxonomy is 97.7–98.4% determined by the kind, depending on the corpus.

The same controls also limit the result. A rule that scores a function by its length has a higher mean AUC than every network we trained, on BigVul a classifier that reads only formatting does better still, the gain over text largely disappears within bins of function size, a fine-tuned pretrained model beats the graph views by only a small margin once formatting is removed, and the labelling function is less accurate class by class than our first, non-blind validation suggested. We therefore recommend that studies of learned vulnerability

detectors report a function-size baseline and a vocabulary-free baseline next to any new architecture or representation, and that proposed abstractions over a parse tree be checked against the grammar alphabet before they are credited with an effect. Cross-project vulnerability detection from small, from-scratch models remains an open problem.

CRediT authorship contribution statement

Author contributions are stated on the title page, which is submitted separately in accordance with the journal's double-blind review policy.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

All code, the annotated gold set of call-site labels, and the scripts that regenerate every table and figure in this paper are publicly available. The repository URL is withheld here to preserve anonymity during review and is given on the title page; it will be included in the camera-ready version. The study uses only publicly available corpora, accessed through the public mirrors identified in Section 4.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used generative AI-assisted tools in order to improve the language and readability of the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, IEEE Transactions on Software Engineering 48 (9) (2022) 3280–3296. doi:10.1109/TSE.2021.3087402.

[2] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, S. McIntosh, Revisiting the performance of deep learning-based vulnerability detection on realistic datasets, IEEE Transactions on Software Engineering 50 (8) (2024) 2163–2177, arXiv:2407.03093. doi:10.1109/TSE.2024.3423712.

[3] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, in: Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), IEEE, 2025, pp. 469–481, arXiv:2403.18624. doi:10.1109/ICSE55347.2025.00038.

[4] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), Internet Society, 2018. doi:10.14722/ndss.2018.23158.

[5] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Transactions on Dependable and Secure Computing 19 (4) (2022) 2244–2258. doi:10.1109/TDSC.2021.3051427.

[6] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: Proceedings of the IEEE Symposium on Security and Privacy (S&P), IEEE, 2014, pp. 590–604. doi:10.1109/SP.2014.44.

[7] Y. Zhou, S. Liu, J. K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, arXiv:1909.03496.

[8] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural networks, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (3) (2021) 1–33. doi:10.1145/3436877.

[9] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2021, pp. 292–303. doi:10.1145/3468264.3468597.

[10] Y. Luo, W. Xu, D. Xu, Detecting code vulnerabilities with heterogeneous GNN training, International Journal of Information Security 24, arXiv:2502.16835 (2025). doi:10.1007/s10207-025-01132-x.

[11] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 608–620. doi:10.1145/3524842.3527927.

[12] D. Hin, A. Kan, H. Chen, M. A. Babar, LineVD: Statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 596–607. doi:10.1145/3524842.3527949.

[13] R. Liu, Y. Wang, H. Xu, J. Sun, F. Zhang, P. Li, Z. Guo, Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection, Information Fusion 115 (2025) 102748, arXiv:2404.14719. doi:10.1016/j.inffus.2024.102748.

[14] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), ACM, 2023, pp. 654–668. doi:10.1145/3607199.3607242.

[15] S. Liu, G. Lin, L. Qu, J. Zhang, O. D. Vel, P. Montague, Y. Xiang, CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Transactions on Dependable and Secure Computing 19 (1) (2022) 438–451. doi:10.1109/TDSC.2020.2984505.

[16] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Transactions on Software Engineering 49 (8) (2023) 4152–4168. doi:10.1109/TSE.2023.3285910.

[17] X. Li, Y. Xin, H. Zhu, Y. Yang, Y. Chen, Cross-domain vulnerability detection using graph embedding and domain adaptation, Computers & Security 125 (2023) 103017. doi:10.1016/j.cose.2022.103017.

[18] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, Journal of Systems and Software 213 (2024) 112038. doi:10.1016/j.jss.2024.112038.

[19] H. Yamamoto, D. Wang, G. K. Rajbahadur, M. Kondo, Y. Kamei, N. Ubayashi, Towards privacy preserving cross project defect prediction with federated learning, in: Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2023, pp. 485–496. doi:10.1109/SANER56733.2023.00052.

[20] Y. Yang, X. Hu, Z. Gao, J. Chen, C. Ni, X. Xia, D. Lo, Federated learning for software engineering: A case study of code clone detection and defect prediction, IEEE Transactions on Software Engineering 50 (2) (2024) 296–321. doi:10.1109/TSE.2023.3347898.

[21] C. Zhang, T. Yu, B. Liu, Y. Xin, Vulnerability detection based on federated learning, Information and Software Technology 167 (2024) 107371. doi:10.1016/j.infsof.2023.107371.

[22] P. Zhou, M. Hu, X. Xie, Y. Liu, M. Chen, VulFL: An empirical study of vulnerability detection using federated learning, arXiv preprint-ArXiv:2411.16099 (2024).

[23] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, C. Zhang, FedProto: Federated prototype learning across heterogeneous clients, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, 2022, pp. 8432–8440. doi:10.1609/aaai.v36i8.20819.

[24] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2016, pp. 308–318. doi:10.1145/2976749.2978318.

[25] I. Mironov, K. Talwar, L. Zhang, Rényi differential privacy of the sampled gaussian mechanism, in: arXiv preprint, 2019, arXiv:1908.10530.

[26] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories (MSR), ACM, 2020, pp. 508–512. doi:10.1145/3379597.3387501.

[27] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[28] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[29] J. Cohen, A coefficient of agreement for nominal scales, Educational and Psychological Measurement 20 (1) (1960) 37–46.

[30] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212.

[31] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, UniXcoder: Unified cross-modal pre-training for code representation, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), 2022, pp. 7212–7225.

Table 5: Hyper-parameters. All values are fixed in advance and shared by every compared system; none is tuned on any fold.

Item	Value
Embedding width / hidden width	64 / 128
Message-passing layers	2 (GGNN: 2 gated steps)
Transformer	2 layers, 4 heads, feed-forward 2× width, dropout 0.1
Readout and head	mean ‖ max pooling; MLP $2h \to h/2 \to 1$, dropout 0.3
Optimiser	Adam, learning rate 0.001
Batch size	64
Epochs	10 (fixed; no early stopping, no validation split)
Loss	binary cross-entropy, positive weight $= \min(n_{\text{neg}}/n_{\text{pos}}, 20)$
Seeds	initialisation and shuffling seed $1000 + k$ for fold k, shared by all systems
Lexical dictionary	up to 9,999 most frequent training-fold tokens with count ≥ 2, plus `<UNK>`
Node cap	200 named AST nodes (pre-order)
Sample	10,000 functions per corpus, uniform at random, seed 0
Folds	$5 \times 2 = 10$ project-level folds
Pretrained reference	UniXcoder-base, 256 tokens, frozen; logistic regression $C = 0.1$, balanced

Table 6: RQ1: ϕ as a static analysis, scored against the consensus of two blind annotators and a third pass on disagreements, pooled over the three corpora. Brackets are 95% Wilson intervals; tp/fp/fn are counts of distinct callee names. The last row scores the question ϕ_8 actually answers (memory or string operation vs. not).

Class	Precision	Recall	tp/fp/fn
`MEMORY_ALLOC`	0.88 [0.75, 0.94]	0.78 [0.65, 0.87]	42/6/12
`MEMORY_REALLOC`	0.97 [0.85, 0.99]	0.94 [0.82, 0.98]	34/1/2
`MEMORY_FREE`	0.85 [0.73, 0.93]	0.95 [0.85, 0.99]	41/7/2
`MEMORY_COPY`	0.63 [0.46, 0.77]	0.88 [0.70, 0.96]	22/13/3
`MEMORY_SET`	1.00 [0.83, 1.00]	0.95 [0.76, 0.99]	19/0/1
`STRING_COPY`	1.00 [0.90, 1.00]	0.82 [0.68, 0.91]	33/0/7
`STRING_CONCAT`	0.97 [0.83, 0.99]	0.93 [0.79, 0.98]	28/1/2
`STRING_FORMAT`	0.86 [0.71, 0.94]	0.88 [0.73, 0.95]	30/5/4
`STRING_LENGTH`	0.97 [0.83, 0.99]	0.97 [0.83, 0.99]	29/1/1
`IO_CALL`	0.86 [0.74, 0.93]	0.69 [0.57, 0.79]	43/7/19
Macro	0.90	0.88	
Memory operation (any of the 9)	0.94	0.94	293/19/19

Table 7: RQ1: annotation agreement and per-corpus scores. Agreement and Cohen's κ_C are between the two blind annotators before adjudication; precision and recall are macro-averaged over classes against the consensus; the last column is precision / recall for "memory or string operation".

Corpus	Names	Agreement	κ_C	Macro P	Macro R	Memory-op P / R
Devign	212	0.92	0.89	0.90	0.87	0.90 / 0.93
BigVul	227	0.94	0.93	0.90	0.87	0.95 / 0.95
DiverseVul	236	0.92	0.90	0.89	0.89	0.96 / 0.94

Table 8: RQ2: node views on BigVul, GGNN, 10 matched project-level folds (mean ± population std over folds). Differences are paired against the lexical baseline: each Δ cell reads mean per-fold difference, 95% cluster-bootstrap interval over held-out projects, bootstrap p / Wilcoxon signed-rank p over folds; *n.s.* marks an interval containing zero. *AUC in size bins* is the AUC computed within five bins of function size (0.5 = no information beyond length). AUPRC is reported without paired tests.

Node view	Folds	AUC	ΔAUC [95% CI] p_{boot} / p_{W}	AUC in size bins	AUPRC
Trivial (all-positive)	—	0.500	—	0.500	0.071
Function size only (rule)	10	0.736 ± 0.065	—	0.500	0.232 ± 0.087
Lexical token	10	0.506 ± 0.077	—	0.467	0.085 ± 0.034
Anonymised token	10	0.708 ± 0.046	+0.202 [+0.149, +0.249] < 0.001 / 0.002	0.569	0.167 ± 0.048
Operation type ϕ_8	10	0.649 ± 0.052	+0.143 [+0.092, +0.211] < 0.001 / 0.010	0.497	0.117 ± 0.026
Grammar kind κ	10	0.691 ± 0.037	+0.185 [+0.130, +0.236] < 0.001 / 0.002	0.535	0.149 ± 0.027
Operation + kind (κ+ϕ)	10	0.687 ± 0.060	+0.181 [+0.132, +0.236] < 0.001 / 0.002	0.536	0.158 ± 0.042
Lexical + kind	10	0.592 ± 0.072	+0.086 [+0.038, +0.111] < 0.001 / 0.002	0.519	0.105 ± 0.039
Anonymised + kind	10	0.701 ± 0.059	+0.195 [+0.144, +0.244] < 0.001 / 0.002	0.558	0.180 ± 0.059

Table 9: Direct paired contrasts between node views (GGNN, 10 matched folds): ΔAUC of the first view minus the second, with 95% cluster-bootstrap interval, bootstrap p / Wilcoxon p. These are the comparisons a reader needs to judge whether one view is better than another; a difference between two views' gaps to the lexical baseline is not a test of the difference.

Contrast	BigVul	DiverseVul
κ+ϕ vs κ	−0.004 [−0.017, +0.021] 0.920 n.s. / 1.000	+0.008 [−0.005, +0.022] 0.232 n.s. / 0.027
κ vs lexical+κ	+0.100 [+0.064, +0.156] < 0.001 / 0.002	+0.060 [+0.030, +0.088] < 0.001 / 0.002
κ vs anonymised+κ	−0.010 [−0.038, +0.014] 0.380 n.s. / 0.625	−0.012 [−0.030, +0.002] 0.098 n.s. / 0.322
anonymised+κ vs lexical+κ	+0.110 [+0.079, +0.166] < 0.001 / 0.002	+0.071 [+0.043, +0.102] < 0.001 / 0.004
lexical+κ vs lexical	+0.086 [+0.038, +0.111] < 0.001 / 0.002	+0.042 [+0.021, +0.061] < 0.001 / 0.037
anonymised vs lexical	+0.202 [+0.149, +0.249] < 0.001 / 0.002	—
κ vs ϕ_8	+0.042 [−0.004, +0.067] 0.072 n.s. / 0.049	—
9-dim bottleneck κ vs κ	+0.017 [+0.004, +0.040] 0.018 / 0.322	—

Table 10: Per-project AUC (BigVul, GGNN) for the held-out projects with at least 10 vulnerable functions, pooled over the folds in which they are tested. It shows that the pooled result is not carried by a single repository.

Project	Functions	Vuln.	Lexical	Lex+κ	κ	κ+ϕ
Android	868	120	0.46	0.51	0.61	0.55
ImageMagick	270	42	0.63	0.73	0.67	0.67
php-src	174	32	0.49	0.63	0.65	0.63
krb5	90	16	0.42	0.57	0.76	0.72
openssl	230	16	0.58	0.70	0.80	0.80
FFmpeg	200	14	0.49	0.61	0.80	0.76
tcpdump	72	14	0.68	0.72	0.64	0.65
ghostscript	214	12	0.43	0.63	0.84	0.87

Table 11: RQ2b: information shared between the operation taxonomy and the grammar kind, computed exactly over every AST node of the corpora indicated. Brackets are 95% bootstrap intervals resampling functions. $H(\phi \mid \kappa)$ is what the taxonomy adds to the grammar kind, and $H(\kappa \mid \phi)$ is what the taxonomy throws away.

	BigVul	Devign	DiverseVul
Sample	10,000 fn / 918,269 nodes	10,000 fn / 1,356,164 nodes	10,000 fn / 1,082,049 nodes
$H(\phi)$	1.449	1.504	1.477
$H(\kappa)$	4.499	4.430	4.507
What the taxonomy adds to the grammar kind			
$H(\phi \mid \kappa)$	**0.053** $[0.052, 0.054]$	**0.060** $[0.060, 0.061]$	**0.061** $[0.060, 0.062]$
$I(\phi;\kappa)/H(\phi)$	96.3%	96.0%	95.9%
ϕ predictable from κ	**98.4%**	**97.7%**	**98.0%**
What the taxonomy discards from the grammar kind			
$H(\kappa \mid \phi)$	**3.103** $[3.093, 3.114]$	**2.987** $[2.975, 2.998]$	**3.091** $[3.080, 3.101]$
$H(\kappa \mid \phi)/H(\kappa)$	69.0%	67.4%	68.6%

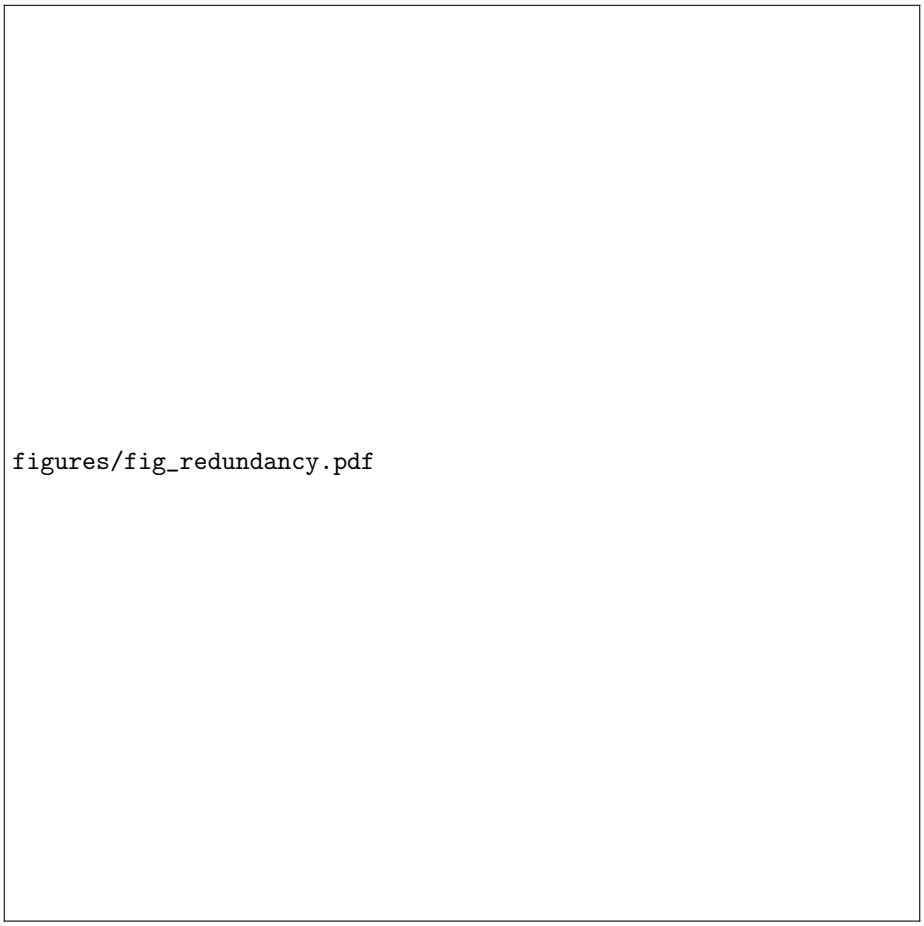


Figure 2: The information the taxonomy carries beyond the grammar kind, $H(\phi \mid \kappa)$ (left), and the share of nodes whose operation label is recoverable from the kind alone (right).

Table 12: RQ3: mean AUC over 10 matched folds for four node views under every backbone and corpus, and the paired gain of κ and of lexical+κ over the lexical view (point estimate, 95% cluster-bootstrap interval, bootstrap p; *n.s.*: interval contains zero).

Corpus	Backbone	Lexical	Lex+κ	κ	κ+ϕ	κ vs lexical	Lex+κ vs lexical
BigVul	GGNN	0.506	0.592	0.691	0.687	$+0.185$ $[+0.130, +0.236] < 0.001$	$+0.086$ $[+0.038, +0.111] < 0.001$
BigVul	GAT	0.507	—	0.673	0.654	$+0.167$ $[+0.117, +0.227] < 0.001$	—
BigVul	GIN	0.574	—	0.704	0.719	$+0.131$ $[+0.089, +0.197] < 0.001$	—
BigVul	Transformer	0.582	—	0.706	—	$+0.124$ $[+0.088, +0.166] < 0.001$	—
DiverseVul	GGNN	0.566	0.607	0.667	0.675	$+0.101$ $[+0.070, +0.132] < 0.001$	$+0.042$ $[+0.021, +0.061] < 0.001$
DiverseVul	GAT	0.558	—	0.652	0.654	$+0.094$ $[+0.060, +0.127] < 0.001$	—
DiverseVul	GIN	0.574	—	0.679	0.687	$+0.105$ $[+0.069, +0.136] < 0.001$	—

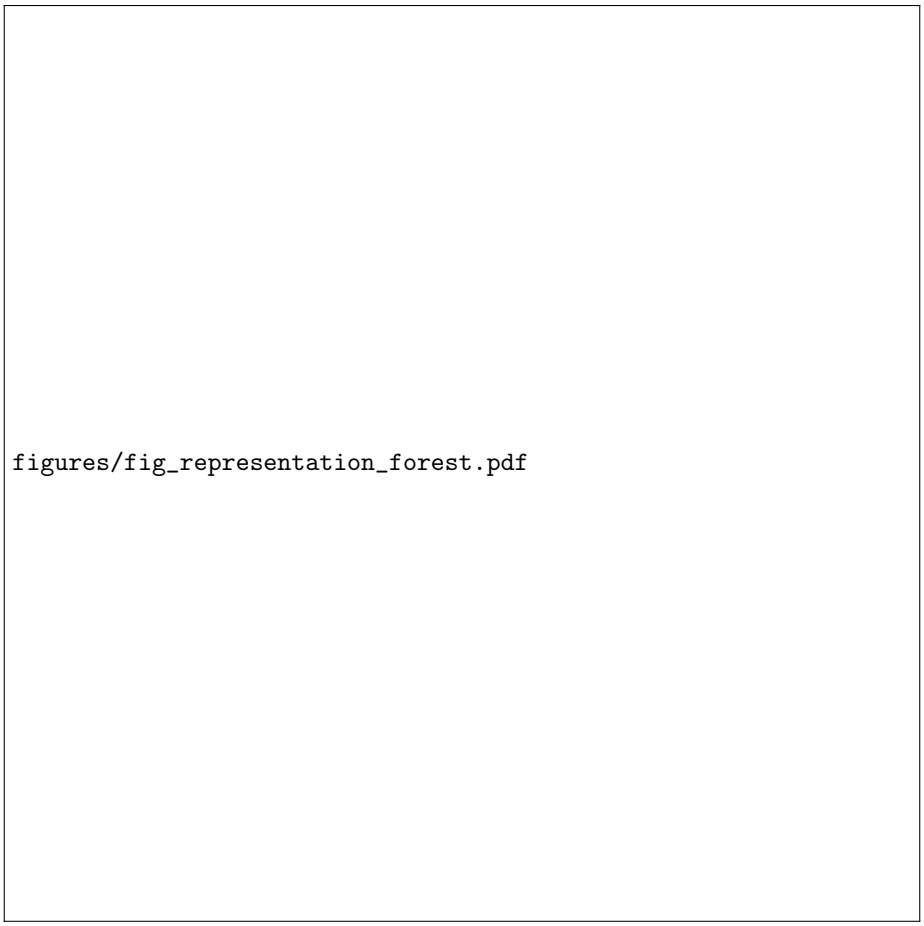


Figure 3: Change in AUC against text-only node features for each corpus and architecture, with 95% cluster-bootstrap intervals over held-out projects. Intervals that include zero would be marked n.s.

Table 13: Ablations on BigVul (GGNN). First block: removing edge types from the graph under the κ view. Second block: refining the operation alphabet under the $\kappa{+}\phi$ view, compared against the same view at $|\mathcal{T}| = 8$ on the same folds. ΔAUC is paired, with 95% cluster-bootstrap interval and bootstrap p.

Variant	Folds	AUC	ΔAUC vs reference [95% CI] p
All edges (AST+SIB+DU), κ	10	0.691	—
no def-use edges	10	0.691	$+0.000$ $[-0.012, +0.019]$ 0.758 n.s. / 1.000
AST edges only	10	0.699	$+0.008$ $[-0.009, +0.027]$ 0.330 n.s. / 0.375
$\kappa{+}\phi$ with $\lvert\mathcal{T}\rvert = 16$ vs 8	10	0.691	$+0.004$ $[-0.022, +0.019]$ 0.920 n.s. / 0.625

Table 14: Comparison with existing approaches re-implemented under the same sample, folds and metrics (mean $\pm$ std over 10 project-level folds). These are re-implementations of the *input representation* of each approach inside one training protocol, not reproductions of the published systems or their published numbers. UniXcoder rows use a frozen encoder and a linear probe. *Format-normalised* source is the parser's token stream with whitespace and comments removed (Section 4.13). The last three rows score functions without reading what the code says.

	BigVul		DiverseVul	
System	AUC	AUPRC	AUC	AUPRC
GGNN on code text (token-only baseline)	0.506 ± 0.077	0.085 ± 0.034	0.566 ± 0.039	0.080 ± 0.017
Devign/ReVeal-style GGNN: text + node type	0.592 ± 0.072	0.105 ± 0.039	0.607 ± 0.027	0.096 ± 0.024
GGNN, anonymised tokens + node type	0.701 ± 0.059	0.180 ± 0.059	0.679 ± 0.036	0.122 ± 0.030
Transformer over node sequence, text	0.582 ± 0.074	0.106 ± 0.042	—	—
Transformer over node sequence, κ	0.706 ± 0.039	0.155 ± 0.041	—	—
Pretrained UniXcoder, frozen, linear probe (raw source)	0.705 ± 0.061	0.163 ± 0.035	0.623 ± 0.027	0.095 ± 0.022
Pretrained UniXcoder, frozen, linear probe (format-normalised)	0.645 ± 0.069	0.109 ± 0.022	0.604 ± 0.015	0.088 ± 0.019
LineVul-style: fine-tuned CodeBERT (raw source)	0.922 ± 0.026	0.748 ± 0.063	0.693 ± 0.028	0.123 ± 0.026
LineVul-style: fine-tuned CodeBERT (format-normalised)	0.728 ± 0.058	0.189 ± 0.058	0.705 ± 0.028	0.131 ± 0.023
No code read: function size (node count)	0.736 ± 0.065	0.232 ± 0.087	0.715 ± 0.028	0.152 ± 0.028
No code read: trailing-whitespace rule	0.830 ± 0.050	0.392 ± 0.109	0.542 ± 0.024	0.069 ± 0.018
No code read: gradient boosting on formatting statistics	0.896 ± 0.026	0.601 ± 0.065	0.634 ± 0.033	0.111 ± 0.034
GGNN, grammar kind κ (ours)	0.691 ± 0.037	0.149 ± 0.027	0.667 ± 0.021	0.105 ± 0.026
GGNN, $\kappa{+}\phi$ (ours)	0.687 ± 0.060	0.158 ± 0.042	0.675 ± 0.021	0.114 ± 0.030